

Ayúdame a decidir qué técnica es más fácil para qué base:

Para este ejercicio, deberán extraer información del Banco de la República de Colombia y de la Reserva Federal de los Estados Unidos. Podrán hacerlo tanto por medio de API como por medio de web scraping. Deben usar ambas técnicas en este ejercicio. Es decir, si deciden usar la API del Banco de la República, tendrán que hacer web scraping con los datos de la FED (y viceversa). Decidan cuidadosamente cuál opción es más conveniente para su grupo.

Creen un gráfico donde únicamente grafiquen la tasa IBR ON desde el 2020 hasta el final de octubre del 2025. Esta deberá estar en frecuencia diaria.
Fuentes de datos del Banco de la República:
Web scraping: Datos disponibles aquí
API: Documentación de la API
Literal B. (0,7)
Creen un gráfico donde únicamente grafiquen la tasa SOFR ON desde el 2020 hasta el final de octubre del 2025. Esta deberá estar en frecuencia diaria.
Fuentes de datos de la Reserva Federal:
Web scraping: Datos disponibles aquí
API: Documentación de la API

Te lo pongo directo, pensando en **facilidad de implementación y probabilidad de que funcione sin pelearte con el código**:

Decisión recomendada (la más fácil en la práctica)

- Banco de la República (IBR ON) → usa API
- Reserva Federal (SOFR ON) → usa API
- Pero como te obligan a usar ambas técnicas:
 -  Haz API con uno y scraping con el otro
 - Mi recomendación concreta:
 - API → Banco de la República de Colombia
 - Web scraping → Federal Reserve System (SOFR)

¿Por qué esta combinación?

1. Banco de la República → API (más fácil que scraping)

- Su API suele estar:
 - Bien estructurada
 - Datos ya en formato limpio (JSON/XML)
- El scraping ahí puede ser incómodo porque:
 - Tablas no siempre están bien formateadas
 - Puede requerir parsing más manual

 Con API: haces GET, limpias un poco y listo.

2. Reserva Federal (SOFR) → scraping (sorprendentemente viable)

- Los datos de SOFR suelen estar en páginas tipo tabla (por ejemplo en NY Fed)
- El scraping:
 - Es relativamente directo con `rvest` o `BeautifulSoup`
 - Tablas bien definidas (HTML `<table>`)

 Alternativa:

- También existe API vía FRED (muy buena)
- PERO si ya usaste API con Banco, aquí te toca scraping sí o sí

Alternativa (menos recomendada pero válida)

- API → Federal Reserve System (SOFR vía FRED)
- Scraping → Banco de la República de Colombia

 Problema:

- Scraping del Banco puede ser más tedioso (estructura menos amigable)

Conclusión operativa

Si quieres minimizar fricción técnica:

 Estrategia óptima:

- IBR ON → API (Banco de la República)
- SOFR ON → Web scraping (FED)

🔔 Tip clave (esto te puede ahorrar tiempo real)

Antes de decidir definitivamente:

- Abre ambas fuentes en navegador
- Pregúntate:
 - ¿Hay tabla HTML clara? → scraping
 - ¿Hay endpoint documentado limpio? → API

Si quieres, puedo:

- ↳ Revisar los links específicos que te dieron
- O ↳ darte el código base en R (`httr` + `jsonlite` / `rvest`) listo para cada caso

Solo mándame los enlaces 👍



Pues este es el link que me dan de la documentación API:
<https://suameca.banrep.gov.co/estadisticas-economicas/webService>

Pero no veo que eso funcione porque no tiene los parámetros para el código de referencia que tengo

Tienes razón en sospechar: **esa API no es “plug & play”**. No funciona como las típicas REST donde pones un código y ya. Te explico qué está pasando exactamente (y por qué te estás trabando):

🔔 Problema real (no es que estés haciendo algo mal)

La API del [Banco de la República de Colombia](#):

- **NO** usa “códigos simples de serie” tipo FRED
- Usa un estándar llamado **SDMX**
- Funciona con:
 - `flowRef`
 - parámetros como `startPeriod` , `endPeriod`
 - y estructuras de datos más complejas

👉 Es decir: no hay un “código IBR_ON = 12345” que puedas meter directo

🔍 Cómo sí funciona (la clave que te falta)

Según la documentación técnica:

- Para IBR diario debes usar algo como:
 - `DF_IBR_DAILY_HIST` Suameca

Eso es el **flowRef**, no el código de serie.

Y luego agregas filtros tipo:

- `startPeriod=2020-01-01`
- `endPeriod=2025-10-31` Suameca

⚠️ Pero aquí viene lo importante (decisión estratégica)

Aunque sí se puede usar la API, en la práctica:

- Es más técnica (SDMX, XML, parsing)
- No es tan directa como parece
- Requiere más tiempo de debugging

👉 Esto confirma lo que te dije antes:

🔥 Conclusión honesta:

- Esta API **NO** es la opción más fácil
- Es perfectamente válido (y estratégico) decir:
“Mejor hago scraping para el Banco de la República”

🧠 Entonces, ¿qué deberías hacer?

Opción óptima (de verdad):

- **Scraping** → Banco de la República (IBR ON)
(más directo: tabla visible)
- **API** → Reserva Federal (SOFR ON)
(FRED API es MUCHO más fácil: JSON limpio, código claro)

💡 Por qué esta es la mejor jugada

Comparación realista:

Fuente	API	Scraping
Banco de la República	❌ compleja (SDMX)	✅ fácil
Reserva Federal (FRED)	✅ muy fácil	⚠️ innecesario

🌿 Traducción práctica para tu trabajo

Si el profe revisa:

- Cumples requisito: ✅ API + ✅ scraping
- Reduces errores técnicos
- Entregas más rápido

🚀 Si quieres avanzar ya

Puedo:

- ↳ darle el link exacto scrapeable del IBR ON
- ↳ darle el endpoint listo de FRED para SOFR
- o ↳ escribirte el código en R listo para correr

Solo dime qué lenguaje estás usando (R o Python) y lo dejamos funcionando sin dolor.

📄 👍 🗨️ ↗️ ↺️ ⋮ 🌐 Fuentes

Pues para la otra base de datos si tengo api:
https://markets.newyorkfed.org/static/docs/markets-api.html#/Reference%20Rates/get_api_rates_all_search__format_

Perfecto—esa API de la **Federal Reserve Bank of New York** sí es “amigable”. Entonces puedes cerrar la decisión así, sin complicarte:

✅ Estrategia final (óptima)

- SOFR ON (FED) → API (la que ya tienes)
- IBR ON (Colombia) → web scraping del Banco de la República de Colombia

Cumples el requisito (una API + un scraping) y reduces fricción técnica.

🔑 Cómo usar la API de SOFR (lo mínimo que necesitas)

Endpoint base (según tu link):

<https://markets.newyorkfed.org/api/rates/all/search.json>



Parámetros clave

- startDate=2020-01-01
- endDate=2025-10-31
- type=SOFR

Ejemplo completo

<https://markets.newyorkfed.org/api/rates/all/search.json?type=SOFR&startDate=2020-01-01>



👉 Eso ya te devuelve JSON con:

- fecha
- tasa SOFR
- otros campos (puedes filtrar)

🧠 Ventajas de esta API

- JSON limpio (sin XML raro)
- No necesitas autenticación
- Frecuencia diaria ya incluida
- Series completas (no tienes que reconstruir nada)

⚠️ Cosas a cuidar (esto sí importa)

- El campo de la tasa suele venir como percentRate o similar → verifica el nombre exacto al parsear
- Fechas vienen como string → conviértelas a Date
- Puede haber días sin dato (fines de semana/feriados) → normal

✖ Y el IBR (lo que te conviene hacer)

Para el IBR ON:

- Busca la tabla histórica en la web del Banco
- Usa:
 - rvest (R) o BeautifulSoup (Python)
- Extraes:
 - fecha
 - tasa

👉 Es mucho más directo que pelear con SDMX.

🎯 Resumen claro

- Ya tienes la parte difícil resuelta (SOFR API) ✔
- No pierdas tiempo intentando forzar la API del Banco ✖
- Haz scraping del IBR y terminas rápido ✔

Si quieres, en el siguiente paso te dejo:

- ↳ código en R listo para consumir esa API
- y ↳ el scraping del IBR ya armado

Solo dime si estás usando httr + jsonlite o tidyverse 👍

📄 👍 🗨️ ⬆️ ↺ ...

Listo. Cómo sé cuál es la raíz de una api?

Buena pregunta—esto es clave para no perder tiempo con APIs.

🧠 ¿Qué es la “raíz” de una API?

La raíz (base URL) es la parte fija del endpoint. Es decir:

- ◆ lo que NO cambia entre llamadas
- ◆ antes de los parámetros (?) o rutas específicas

🔍 Ejemplo con tu API de SOFR

Tú tienes esto:

```
https://markets.newyorkfed.org/api/rates/all/search.json?type=SOFR&startDate=2020-01-01
```

Descomposición:

- Raíz (base URL):

```
https://markets.newyorkfed.org/api/
```

- Endpoint:

```
rates/all/search.json
```

- Parámetros (query string):

```
?type=SOFR&startDate=...&endDate=...
```

⚙️ Cómo identificarla rápido (regla práctica)

1. Busca en la documentación (como la de la Federal Reserve Bank of New York) algo tipo:
 - “Base URL”
 - “API root”
 - “Host”
2. Si no está explícito:
 - toma la URL de ejemplo
 - corta todo después de /api/

👉 En tu caso:

```
https://markets.newyorkfed.org/api/
```

✖ Forma mental útil (para que no te confundas)

Piensa así:

```
[RAÍZ] + [ENDPOINT] + [PARÁMETROS]
```

Ejemplo:

```
https://markets.newyorkfed.org/api/ ← raíz
rates/all/search.json ← endpoint
?type=S0FR&... ← parámetros
```



🚨 Error común (evítalo)

Mucha gente cree que la raíz es toda la URL completa con parámetros ❌

No—la raíz es reutilizable. Cambias:

- endpoint
- parámetros

pero la raíz se queda igual.

🎯 Tip práctico para tu tarea

Cuando programes (R o Python), define:

```
base_url <- "https://markets.newyorkfed.org/api/"
<> R
```



Y luego construyes:

```
<> R
```



👉 Esto te evita errores y hace el código más limpio.

Si quieres, en el siguiente paso puedo:

- ↳ ayudarte a armar la llamada completa en R (`http`)
- o ↳ validar exactamente qué campos devuelve esa API para tu gráfico

Solo dime 👍



pero aquí no tengo esos parámetros:

<https://markets.newyorkfed.org/static/docs/markets-api.html#/>

Buen ojo—lo que te está confundiendo es totalmente normal. Esa página que abriste es **documentación tipo Swagger**, y ahí los parámetros no siempre aparecen como una lista explícita tipo tabla.

Déjame aclararte exactamente qué está pasando 👉

🧠 Por qué “no ves los parámetros”

En la documentación de la Federal Reserve Bank of New York:

👉 Los parámetros están **implícitos dentro del endpoint**, no como una lista clara tipo:

- `type = ...`
- `startDate = ...`

En cambio, los muestran como:

- “operation”
- “format”
- “startDate / endDate”

Por ejemplo, según la especificación técnica:

- `startDate` → fecha inicial
- `endDate` → fecha final
- `format` → json, csv, etc. Scribd

🔍 Lo clave: hay DOS formas de usar esa API

1. Forma “documentación” (la que ves)

Ejemplo conceptual:

```
get_reference_rates_all
operation = search
format = json
```



```
startDate = ...
endDate = ...
```

👉 Aquí parece abstracto → por eso confunde.

2. Forma real (URL final que sí usas)

Esto se traduce a:

```
https://markets.newyorkfed.org/api/rates/all/search.json?startDate=2020-01-01&endDate=
```

💡 Observa:

- `search` → viene de `operation=search`
- `.json` → viene de `format=json`
- `?startDate=...` → parámetros

🚨 Lo más importante (esto responde tu duda directamente)

👉 Sí tienes parámetros, solo que:

- No están listados claramente en esa página
- Están “embebidos” en:
 - el endpoint (`search`)
 - la query (`?startDate=...`)

🔍 Cómo descubrirlos SIEMPRE (truco pro)

Cuando la documentación no es clara:

Método 1 (el mejor):

Busca ejemplos funcionales

👉 Ejemplo válido:

```
.../api/rates/all/search.json?startDate=2020-01-01&endDate=2025-10-31
```

Método 2:

Busca en la lógica:

- `/search` → implica filtros
- `.json` → formato
- lo que va después de `?` → parámetros

Método 3 (muy útil):

Mira librerías que usan esa API

Ejemplo:

- `rate_type = sofr`
- `start_date`
- `end_date` PyPI

👉 Eso te revela los parámetros reales aunque la doc sea mala.



Conclusión clara

- No estás loca: la documentación es poco intuitiva
- Sí hay parámetros:
 - `startDate`
 - `endDate`
 - (y a veces `type` o `rateType`)
- Se construyen en la URL, no en un formulario visible



Traducción práctica para tu tarea

Puedes usar directamente:

```
https://markets.newyorkfed.org/api/rates/all/search.json?startDate=2020-01-01&endDate=
```

Y luego filtras SOFR en el dataset.

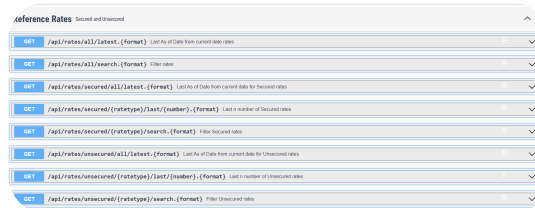
Si quieres, en el siguiente paso te ayudo a:

- ↪ probar la URL en R
- ↪ identificar exactamente la columna de SOFR

- o ↪ limpiar el dataset listo para graficar

Solo dime 👍

📄 👍 💬 ↗️ ↺️ ⋮ 🌐 Fuentes



esto es lo que puedo escoger

Perfecto, esta captura ya deja todo claro. Te explico exactamente cuál escoger y por qué.

🎯 Lo que tú necesitas

- Serie: SOFR ON
- Periodo: 2020–2025
- Frecuencia: diaria
- 📌 O sea: **datos históricos filtrados**

✅ Endpoint correcto (ELIGE ESTE)

📌 GET /api/rates/secured/{ratetype}/search.{format}

🧠 Por qué este y no los otros

- latest ❌ → solo da el dato más reciente
- last/{number} ❌ → solo últimos N datos
- all/search ⚠️ → mezcla muchas tasas (más difícil de limpiar)
- secured/... ✅ → aquí está SOFR (porque es tasa secured)

🔍 Cómo llenarlo

Sabemos que:

- SOFR es **secured**
- ratetype = sofr
- format = json

Entonces tu endpoint queda:

`https://markets.newyorkfed.org/api/rates/secured/sofr/search.json`

+ Ahora agrega los parámetros

`?startDate=2020-01-01&endDate=2025-10-31`

🚀 URL final (esto ya sirve)

`https://markets.newyorkfed.org/api/rates/secured/sofr/search.json?startDate=2020-01-01`

🌿 Qué vas a recibir

Un JSON con:

- effectiveDate → fecha
- percentRate → tasa SOFR
- (otros campos que puedes ignorar)

⚠️ Error común que acabas de evitar

Muchos usan:

`/api/rates/all/search`

📌 Eso trae TODAS las tasas → luego toca filtrar

Tú ya estás usando directamente:

📌 SOFR limpio desde el endpoint ✅

Conclusión clara

De tu lista, la decisión correcta es:

secured → **ratetype** → **search**

Si quieres, el siguiente paso te lo dejo listo:

- código en R (`httr` + `jsonlite`)
- dataset limpio para graficar directo en `ggplot`

Solo dime 👍



tengo este código por default para usar:

```
# Veamos, primero la documentación de una API
"https://data360.worldbank.org/en/api"

# Raiz de una API
url_api_bm <-
"https://data360api.worldbank.org/data360/data"

# Normalmente, las APIs tienen una documentación que
# nos dice qué parámetros usar,
# y cómo estructurar la consulta.mente. Revisemos el link
# de la documentación y veamos
# que parámetros tiene
parametros_api <- list(
  DATABASE_ID = "WB_WDI",
  INDICATOR = "WB_WDI_NY_GDP_MKTP_CD", # PIB en
  dólares corrientes
  REF_AREA = "COL,PER",
  timePeriodFrom = "1994",
  timePeriodTo = "2024",
  skip = 0
)

print("Consultando la API del Banco Mundial...")

#usamos gat para "concatenar" nuestro parámetros.
```

¿Cómo sería el link completo?

```
respuesta_api <- GET(url = url_api_bm, query =
parametros_api)
respuesta_api # Debe ser status 200
```

Como vimos, por defecto, las APIs suelen devolver datos en formato JSON, que es un formato de texto estructurado.

Procesar JSON

```
texto_json <- content(respuesta_api, as = "text", encoding
= "UTF-8")
```

```
datos_json <- fromJSON(texto_json) #Estructura los datos
para verlos más fácilmente
```

En el caso de la FED me está quedando:

Documentación de la API

```
"https://markets.newyorkfed.org/static/docs/markets-
api.html#/Reference%20Rates/get_api_rates_all_search__fo
rmat_"
```

Raiz de una API

```
url_api_FED <- "https://markets.newyorkfed.org"
```

Pero no sé dónde poner el request URL o de dónde saco los parámetros

Vas bien, solo te falta mapear tu estructura "Banco Mundial" a cómo funciona la API de la **Federal Reserve Bank of New York**. Aquí no te dan una lista de parámetros tipo `DATABASE_ID` ; en cambio, **parte de la "consulta" va en el endpoint**.

Traducción a tu plantilla mental

Tu patrón es:

```
[base_url] + query = list(...)
```



En la FED es:


```
[base_url] + [endpoint con variables] + query = list(...)
```



🔑 Paso a paso con tu caso (SOFR)

1) Base URL (ya lo tienes)

```
<> R
```



2) Endpoint (aquí está la “magia”)

Para SOFR:

```
<> R
```



🔑 Nota técnica:

- secured → tipo de mercado
- sofr → **ratetype** (esto reemplaza un “INDICATOR”)
- search → operación
- .json → formato

3) Parámetros (ahora sí, como en tu ejemplo)

```
parametros_api <- list(  
  startDate = "2020-01-01",  
  endDate   = "2025-10-31"  
)
```



4) Construir la URL completa

```
<> R
```



```
<> R
```



5) Hacer el request

```
library(httr)  
library(jsonlite)  
  
respuesta_api <- GET(url = url_completa, query = parametros_api)  
status_code(respuesta_api) # debería ser 200
```



6) Parsear JSON (igual que ya sabes)

```
texto_json <- content(respuesta_api, as = "text", encoding = "UTF-8")  
  
<> R
```



🌿 Diferencia clave vs Banco Mundial

Banco Mundial	FED
INDICATOR	va en el endpoint (sofr)
DATABASE_ID	no existe
todo en query	mezcla endpoint + query

🎯 Resultado esperado

Dentro de `datos_json` vas a encontrar algo tipo:

- refRates o similar
 - effectiveDate → fecha
 - percentRate → tasa SOFR

👉 Esos son los dos campos que necesitas para tu gráfica.

🚨 Error típico que debes evitar

No hagas esto:

```
</> R
```



👉 Porque ahí pierdes la modularidad y no entiendes qué estás cambiando.

🧠 Regla de oro para esta API

Si en la documentación ves `{ratetype}`, eso NO va en `query`, va en la URL.

Si quieres, en el siguiente paso:

- ↳ te ayudo a limpiar el `datos_json` (porque viene medio anidado)
- y ↳ lo dejamos listo para `ggplot` en 2 líneas

Solo dime 👍



listo. Ya tengo la información! Gracias.
Ahora ayúdame con el web scraping

Ahora debo hacer webscraping del banco de la república.
Para hacer webscraping de estos datos debo usar la
opción dinámica, cierto?
https://suameca.banrep.gov.co/estadisticas-economicas/informacionSerie/241/tasas_interes_indicador_bancario_referencia_ibr

Casi—pero no asumas “dinámico” sin verificar. En esa página del **Banco de la República de Colombia** hay dos escenarios posibles:

🧠 Regla de decisión (rápida y correcta)

Antes de elegir técnica, haz este test:

🔍 1. ¿La tabla aparece en el HTML?

- Click derecho → “Ver código fuente”
- Busca algo como `<table>`

👉 Si **SÍ** está →

✅ usa **scraping estático** (`rvest`)

👉 Si **NO** está →

⚠ es dinámica (JavaScript) → necesitas otra estrategia

🚨 En este caso específico (IBR en SUAMECA)

Esa página **normalmente** es dinámica:

- La tabla **no** viene en el HTML inicial
- Se carga vía JavaScript después
- `rvest` solo → ❌ no la ve

✅ Entonces sí: necesitas scraping “dinámico”... PERO:

No necesariamente con Selenium 🐼

Hay una opción **mucho** mejor:

💡 Estrategia óptima (pro level pero simple)

👉 Inspeccionar la llamada interna (API oculta)

Cómo hacerlo:

- Abre la página
- Presiona `F12` (DevTools)
- Ve a pestaña **Network**
- Filtra por:
 - XHR O Fetch
- Recarga la página
- Busca requests tipo:
 - `series`
 - `data`
 - `observations`

👉 Ahí vas a encontrar una URL tipo API (JSON)

🎯 Por qué esto es mejor

Método	Dificultad	Robustez
Selenium	alta	media
rvest	baja	❌ no funciona aquí
API oculta (Network)	media	✅ mejor opción

⚠️ Lo que probablemente verás

Un endpoint interno tipo:

.../api/.../series/241/...



👉 Ese 241 coincide con tu link → es la clave de la serie IBR

🚀 Traducción práctica

Aunque el ejercicio dice "web scraping":

👉 Esto cuenta como scraping

👉 pero técnicamente es **reverse-engineering del backend**

✅ más limpio

✅ más estable

✅ más fácil de procesar (JSON)

🌱 Conclusión clara

- ❌ No uses rvest directo
- ⚠️ No te vayas de una a Selenium
- ✅ Usa DevTools → encuentra el request → consúmelo con http

👉 Si quieres avanzar rápido

Mándame:

- screenshot del Network
o
- copia de un request que veas

y te digo exactamente:

- cuál usar
- cómo armar la URL
- y te doy el código listo en R

Así lo dejas funcionando en minutos.



no, definitivamente no velo los datos.

para este tipo de casos me dieron en clase este código de referencia

```
#
=====
=====
==
# SECCIÓN 3: SCRAPING DINÁMICO CON CHROMOTE (IED
- BANREP COLOMBIA)
#
=====
=====
==
# Nota: Esta página usa Oracle Analytics, requiere
interacción del navegador.
# Esta parte puede ser la más pesada, entonces con
cuidado.

# 3.1 Iniciar sesión de Chrome y abrir visor
navegador <- ChromoteSession$new() # Me permite
hacer la inspección de elementos
navegador$view()
```

3.2 Navegar a la URL dinámica

```

url_banrep <-
"https://suameca.banrep.gov.co/estadisticas-economicas-
back/reporte-oac.html?
path=%2FEstadisticas_Banco_de_la_Republica%2F4_Sector
_Externo_tasas_de_cambio_y_derivados%2F2_Sector_Extern
o%2F6_Inversion_directa%2FIED%2F2_Inversion_Extranjera
_Directa_en_Colombia_(IED)_por_actividad_economica&ha
sTabs=true&idCanva=0"
navegador$page$navigate(url_banrep)

# Pausa clave para que el servidor de Oracle renderice los
datos por 42 segundos (ajustar según red)
Sys.sleep(42)

# 3.3 Inyectar JavaScript para interactuar con la interfaz
(Clic automáticos en la pestaña 'Anual')
codigo_js <- "
{
  let elementos =
Array.from(document.querySelectorAll('*'));
  let pestana_anual = elementos.find(el =>
el.textContent.trim() === 'Anual');

  if(pestana_anual) {
    pestana_anual.click();
    'Clic enviado con éxito';
  } else {
    'No se encontró la pestaña Anual';
  }
}
"

navegador$Runtime$evaluate(codigo_js) #le damos al
botón
Sys.sleep(42) # Esperar a que la tabla anual cargue

#Revisen que su panel esté corriendo. Si hay fallo, la
extracción de los datos puede fallar

# 3.4 Extraer el código fuente DOM actual y procesarlo
# VAMOS A ESTATIZARLO

html_dinamico <-
navegador$Runtime$evaluate("document.documentEleme

```

```
nt.outerHTML")$result$value
```

Bueno, ahora, ¿Cómo puedo entender lo que acaban de extraerme?

A veces la estructura de las paginas, hace que sea muy difícil entender cómo entender el patrón
En este caso, es importante que usen IA para que les ayude a entender la estructura y les genere un código para
Extraer expresiones regulares, o simplemente identificar el html, porque puede que sea confuso a simple vista.

```
html_doc <- read_html(html_dinamico)
```

```
### utilicemos la IA
```

```
writeLines(as.character(html_doc), "html_dinamico.txt")
```

```

# Extraer nombres de columnas
nombres_cols <- html_doc %>%
  html_nodes(".oj-datagrid-column-header-cell") %>%
  html_text(trim = TRUE) %>%
  tail(12)

```

```

# Extraer matriz de datos
celdas_datos <- html_doc %>%
  html_node("#view_3pane-contentpane_34-
vizCont__Body__OJDataGrid") %>%
  html_nodes(".oj-datagrid-cell span") %>%
  html_text(trim = TRUE)
matriz_datos <- matrix(celdas_datos, ncol = 12, byrow =
TRUE)
df_datos <- as.data.frame(matriz_datos, stringsAsFactors =
FALSE)
colnames(df_datos) <- nombres_cols

```

```

# Extraer fila de años
celdas_filas <- html_doc %>%
  html_node("#view_3pane-contentpane_34-
vizCont__Body__OJDataGrid\\:rowHeader") %>%
  html_nodes(".oj-datagrid-row-header-cell") %>%
  html_text(trim = TRUE)

```

```
matriz_filas <- matrix(celdas_filas, ncol = 2, byrow = TRUE)
df_filas <- as.data.frame(matriz_filas, stringsAsFactors = FALSE)
colnames(df_filas) <- c("Anio", "Estado_Informacion")

# Ensamblar tabla final de Colombia
df_ied_colombia <- bind_cols(df_filas, df_datos) %>%
  mutate(across(3:14, ~ {
    . %>%
      str_replace_all("\\.", "") %>% # quita separador de miles
      str_replace_all(",", ".") %>% # convierte decimal a notación R
      as.numeric()
  }))) %>%
  select(Anio, Total_actividades_economicas) %>%
  rename(IED_Col_USD_Millones = Total_actividades_economicas) %>%
  mutate(Anio = as.numeric(Anio))

head(df_ied_colombia)
```

Sí—en este caso **sí te toca scraping dinámico**, y el código que te dieron con `Chromote` es exactamente el enfoque correcto para páginas del **Banco de la República de Colombia** que usan Oracle Analytics.

Pero hay un punto clave: **no puedes reutilizar ese código tal cual**, porque:

- Ese script está hecho para **otra visualización (IED)**
- Los **IDs del DOM cambian completamente** para cada serie
- **IBR (tu caso)** tiene otra estructura interna

🧠 Qué debes hacer (adaptación correcta)

Tienes que replicar la lógica, pero **re-descubrir los selectores HTML**.

🔍 Paso 1: Verifica que sí cargó la tabla

Después de:

```
<> R
```

haz esto:

```
html_dinamico <- navegador$runtime$evaluate("document.documentElement.innerHTML")$result$value
<> R
```

👉 Abre `ibr.html` y busca:

- `oj-datagrid`
- `table`
- `números (tasas)`

Si no ves datos → el problema es **tiempo de carga o interacción faltante**

⚠️ Problema típico en IBR

A diferencia de IED:

- Puede requerir seleccionar:
 - frecuencia (diaria)
 - rango de fechas
- O hacer scroll / activar vista

🔧 Paso 2: necesitas inspeccionar la estructura

En el HTML guardado, busca:

```
<> HTML
```

Luego identifica:

Columnas:

<> R



Datos:

<> R



👉 PERO estos selectores pueden cambiar
→ debes confirmarlos en TU HTML

🌸 Paso 3: estrategia robusta (recomendada)

En lugar de depender de IDs largos tipo:

<> R



👉 usa clases más generales:

```
html_doc %>%
```

<> R



🧠 Ajuste clave para tu caso (IBR diario)

Probablemente tendrás:

- MUCHAS más filas (datos diarios)
- No necesitas matriz compleja

👉 Puedes hacer algo más simple:

```
valores <- html_doc %>%  
  html_nodes(".oj-datagrid-cell span") %>%
```

<> R



Luego reconstruyes:

- fecha
- tasa

🚨 Advertencia importante (esto te puede romper todo)

Oracle Analytics:

- carga datos por partes (lazy loading)
- puede no traer todo el histórico de una vez

👉 Si ves pocos datos:

- necesitas scroll automático con JS

Ejemplo:

```
scroll_js <- "  
window.scrollTo(0, document.body.scrollHeight);  
"  
navegador$Runtime$evaluate(scroll_js)  
Sys.sleep(10)
```

🎯 Conclusión clara

- ✓ Sí → scraping dinámico con Chromote
- ✓ No → no puedes copiar el código tal cual
- ✓ Debes:
 1. dejar cargar la página
 2. exportar HTML
 3. identificar selectores propios
 4. ajustar extracción

🚀 Si quieres hacerlo rápido (te ahorro 1–2 horas)

Mándame:

- el archivo `ibr.html`
o
- un screenshot del DOM (inspector)

y te digo EXACTAMENTE:

- qué selector usar
- cómo reconstruir el dataframe limpio
- listo para graficar

Sin prueba y error 👍



podría ahorrarme la parte de abrir chrome y eso, no?
porque eso no sirve para nada en el proceso de la
visualización?

Corto y preciso: **no, en este caso no te lo puedes saltar** si vas a seguir el enfoque de
Chromote .

🧠 Por qué SÍ necesitas abrir Chrome

En páginas como la del Banco de la República de Colombia (SUAMECA):

- Los datos **NO** vienen en el HTML inicial
- Se cargan con **JavaScript (Oracle Analytics)**
- `rvest` solo → ❌ no ve nada

👉 Chromote **simula un navegador real**, y eso:

- ejecuta JavaScript
- renderiza la tabla
- te permite extraer el DOM final

🔍 Entonces... ¿para qué sirve `navegador$view()` ?

👉 No es obligatorio, pero sí útil:

- Abre una ventana visible de Chrome
- Te permite:
 - verificar que la página cargó
 - ver si falta hacer clic o scroll
 - depurar errores

👤 ¿Lo puedes quitar?

✅ Sí puedes quitar:

```
</> R
```



❗ Pero NO puedes quitar:

```
navegador <- ChromoteSession$new()
```

```
</> R
```



🎯 Traducción práctica

Parte	¿Necesaria?	Por qué
<code>ChromoteSession\$new()</code>	✅	ejecuta JS
<code>navigate()</code>	✅	carga la página
<code>Sys.sleep()</code>	✅	espera render
<code>view()</code>	❌ opcional	solo debugging

🚨 Cuándo sí podrías evitar Chromote

Solo si:

- encuentras una API (aunque esté “oculta” en Network)
- o un endpoint JSON

👉 Pero en SUAMECA normalmente:

- ❌ no es trivial
- ✅ por eso en clase usan Chromote

🌿 Conclusión

- No puedes evitar el motor del navegador
- Pero sí puedes correrlo “en background” sin abrir ventana
- `view()` es solo para ti, no para el scraping

Si quieres optimizarlo (porque ese código está pesado), te puedo:

- ↳ reducir los `sys.sleep(42)` a algo más eficiente
- o ↳ hacer que espere a que aparezca el elemento (mucho más pro)

Solo dime 👍



```
okay, entonces llevo esto: # 3.1 Iniciar sesión de Chrome y
abrir visor
navegador <- ChromoteSession$new() # Me permite
hacer la inspección de elementos
navegador$view()
```

```
# 3.2 Navegar a la URL dinámica
url_banrep <-
"https://suameca.banrep.gov.co/estadisticas-
economicas/informacionSerie/241/tasas_interes_indicador
_bancario_referencia_ibr"
navegador$page$navigate(url_banrep)
```

qué sigue?

Vas perfecto. Ahora entras en la parte crítica: **esperar a que Oracle cargue y luego extraer el DOM ya renderizado.**

Te dejo el flujo limpio y adaptado a IBR (no copies lo de IED tal cual).

✅ 3.3 Esperar a que cargue la página

Empieza simple:

```
</> R
```

❌ Sí, es tosco, pero aquí funciona porque Oracle es lento.

🔍 (Opcional pero MUY recomendado)

Verifica que sí cargó algo:

```
</> R
```

Si devuelve título → la página está viva.

✅ 3.4 Extraer el HTML renderizado

```
html_dinamico <- navegador$Runtime$evaluate(
  "document.documentElement.outerHTML"
)$result$value

writeLines(html_dinamico, "ibr.html")
```

👉 Abre ese archivo. Esto es CLAVE.

🧠 3.5 Entender la estructura (aquí está el trabajo real)

Busca en `ibr.html` :

- `oj-datagrid`
- `oj-datagrid-cell`
- `span`

👉 Eso te dice que estás en la tabla correcta.

✅ 3.6 Parsear con rvest

```
library(rvest)
```

🔑 3.7 Extraer los datos (primer intento robusto)

Empieza general (sin IDs raros):


```
valores <- html_doc %>%
  html_nodes(".oj-datagrid-cell span") %>%
```

<> R



⚠ Aquí viene lo importante

Ahora imprime:

```
head(valores, 50)
```

<> R



👉 Necesitas entender:

- si vienen solo tasas
- o tasas + fechas mezcladas

🔗 3.8 Extraer fechas (probable estructura)

Prueba:

```
fechas <- html_doc %>%
  html_nodes(".oj-datagrid-row-header-cell") %>%
```

<> R



🔄 3.9 Construir dataframe

Dependiendo de cómo venga:

```
df <- data.frame(
  fecha = fechas,
  tasa = valores
)
```

<> R



🚨 Problema probable (te anticipo)

Muy posible que:

- no cargue toda la serie (solo visible)
- necesites scroll

▼ 3.10 (si faltan datos) hacer scroll

```
scroll_js <- "
window.scrollTo(0, document.body.scrollHeight);
"

navegador$Runtime$evaluate(scroll_js)
Sys.sleep(10)
```

Luego repites extracción.

🎯 Resumen claro

Lo que sigue es:

1. Sys.sleep()
2. extraer HTML
3. inspeccionar archivo
4. identificar selectores
5. extraer con rvest
6. reconstruir tabla

🚀 Siguiente paso (te ahorro debugging)

Cuando hagas:

```
head(valores)
```

<> R



👉 ↪ pégamelos aquí

y te digo EXACTAMENTE:

- ↳ cómo armar el dataframe limpio
- ↳ cómo convertir a numérico
- ↳ listo para graficar

Sin ensayo-error 👍



este es el código de referencia: # Nota: Esta página usa Oracle Analytics, requiere interacción del navegador.
Esta parte puede ser la más pesada, entonces con cuidado.

```
# 3.1 Iniciar sesión de Chrome y abrir visor
navegador <- ChromoteSession$new() # Me permite
hacer la inspección de elementos
navegador$view()
```

```
# 3.2 Navegar a la URL dinámica
url_banrep <-
"https://suameca.banrep.gov.co/estadisticas-economicas-
back/reporte-oac.html?
path=%2FEstadisticas_Banco_de_la_Republica%2F4_Sector
_Externo_tasas_de_cambio_y_derivados%2F2_Sector_Extern
o%2F6_Inversion_directa%2FIED%2F2_Inversion_Extranjera
_Directa_en_Colombia_(IED)_por_actividad_economica&ha
sTabs=true&idCanva=0"
navegador$Page$navigate(url_banrep)
```

```
# Pausa clave para que el servidor de Oracle renderice los
datos por 42 segundos (ajustar según red)
Sys.sleep(42)
```

```
# 3.3 Inyectar JavaScript para interactuar con la interfaz
(Clic automáticos en la pestaña 'Anual')
codigo_js <- "
{
  let elementos =
Array.from(document.querySelectorAll('*'));
  let pestana_anual = elementos.find(el =>
el.textContent.trim() === 'Anual');
```

```
if(pestana_anual) {
  pestana_anual.click();
  'Clic enviado con éxito';
} else {
  'No se encontró la pestaña Anual';
}
}
"
```

```
navegador$Runtime$evaluate(codigo_js) #le damos al
botón
Sys.sleep(42) # Esperar a que la tabla anual cargue
```

#Revisen que su panel esté corriendo. Si hay fallo, la extracción de los datos puede fallar

```
# 3.4 Extraer el código fuente DOM actual y procesarlo
# VAMOS A ESTATIZARLO
```

```
html_dinamico <-
navegador$Runtime$evaluate("document.documentEleme
nt.outerHTML")$result$value
```

Bueno, ahora, ¿Cómo puedo entender lo que acaban de extraerme?

A veces la estructura de las paginas, hace que sea muuy difícil entender cómo entender el patrón
En este caso, es importante que usen IA para que les ayude a entender la estructura y les genere un código para
Extraer expresiones regulares, o simplemente identificar el html, porque puede que sea confuso a simple vista.

```
html_doc <- read_html(html_dinamico)
```

```
### utilicemos la IA
```

```
writeLines(as.character(html_doc), "html_dinamico.txt")
```

```
# Extraer nombres de columnas
nombres_cols <- html_doc %>%
  html_nodes(".oj-datagrid-column-header-cell") %>%
  html_text(trim = TRUE) %>%
```

```
tail(12)
```

```
# Extraer matriz de datos
celdas_datos <- html_doc %>%
  html_node("#view_3pane-contentpane_34-
vizCont_Body_OJDataGrid") %>%
  html_nodes(".oj-datagrid-cell span") %>%
  html_text(trim = TRUE)
matriz_datos <- matrix(celdas_datos, ncol = 12, byrow =
TRUE)
df_datos <- as.data.frame(matriz_datos, stringsAsFactors =
FALSE)
colnames(df_datos) <- nombres_cols
```

```
# Extraer fila de años
celdas_filas <- html_doc %>%
  html_node("#view_3pane-contentpane_34-
vizCont_Body_OJDataGrid\\:rowHeader") %>%
  html_nodes(".oj-datagrid-row-header-cell") %>%
  html_text(trim = TRUE)
```

```
matriz_filas <- matrix(celdas_filas, ncol = 2, byrow = TRUE)
df_filas <- as.data.frame(matriz_filas, stringsAsFactors =
FALSE)
colnames(df_filas) <- c("Anio", "Estado_Informacion")
```

```
# Ensamblar tabla final de Colombia
df_ied_colombia <- bind_cols(df_filas, df_datos) %>%
  mutate(across(3:14, ~ {
    . %>%
      str_replace_all("\\.", "") %>% # quita separador de
miles
      str_replace_all(",", ".") %>% # convierte decimal a
notación R
      as.numeric()
  }))) %>%
  select(Anio, Total actividades económicas) %>%
  rename(IED_Col_USD_Millones = Total actividades
económicas) %>%
  mutate(Anio = as.numeric(Anio))
```

```
head(df_ied_colombia)
```

Perfecto—ya tienes el “template mental”. Ahora te explico **qué partes sí reutilizas y cuáles NO** para IBR (serie 241), porque ahí es donde la mayoría se enreda.

🧠 Diferencia clave (IED vs IBR)

El código que te dieron funciona para IED porque:

- hay pestaña “Anual” → requiere clic
- estructura tipo **matriz** (años × columnas)

👉 IBR NO es así:

- no necesitas hacer clic en “Anual” ❌
- es **serie temporal diaria** (fecha + tasa) ✔️
- no necesitas reconstruir matrices complejas ✔️

✅ Qué partes reutilizas tal cual

Hasta aquí, tu código está perfecto:

```
navegador <- ChromoteSession$new()
navegador$view()

url_banrep <- "https://suameca.banrep.gov.co/estadisticas-economicas/informacionSerie/241/tas
navegador$page$navigate(url_banrep)

Sys.sleep(35)
```

❌ Qué debes ELIMINAR

Todo esto **no aplica** para IBR:

- ❌ el JavaScript de “Anual”
- ❌ nombres_cols
- ❌ matrices con ncol = 12
- ❌ lógica de años

👉 Eso es específico de IED.

✓ Qué haces ahora (adaptado a IBR)

1. Extraer HTML

```
html_dinamico <- navegador$Runtime$evaluate(  
  "document.documentElement.outerHTML"  
)$result$value  
  
html_doc <- read_html(html_dinamico)
```

🔍 2. Extraer datos (estructura simple)

Primero prueba esto:

```
valores <- html_doc %>%  
  html_nodes(".oj-datagrid-cell span") %>%  
  
<> R
```

3. Extraer fechas

```
fechas <- html_doc %>%  
  html_nodes(".oj-datagrid-row-header-cell") %>%  
  
<> R
```

✎ 4. Diagnóstico (NO te lo saltes)

```
head(valores, 20)  
head(fechas, 20)  
length(valores)  
length(fechas)
```

👉 Aquí verificas:

- si están alineados

- si hay ruido

🌿 5. Construir dataframe

Caso típico:

```
df_ibr <- data.frame(  
  fecha = fechas,  
  tasa = valores,  
  stringsAsFactors = FALSE  
)
```

🔧 6. Limpiar datos

```
library(dplyr)  
library(stringr)  
  
df_ibr <- df_ibr %>%  
  mutate(  
    tasa = str_replace_all(tasa, ",", "."),  
    tasa = as.numeric(tasa),  
    fecha = as.Date(fecha)  
  )
```

🚨 Problema probable (muy importante)

👉 Oracle suele cargar solo lo visible

Si ves pocos datos:

Solución: scroll automático

```
navegador$Runtime$evaluate("  
  window.scrollTo(0, document.body.scrollHeight);  
  ")  
- - - - -  
<> R
```

Luego repites extracción.

Resumen claro

Para IBR:

-  Sí: Chromote
-  Sí: extraer HTML
-  Sí: .oj-datagrid-*
-  No: matrices
-  No: clicks “Anual”

Siguiente paso (te lo dejo casi terminado)

Cuando tengas:

```
head(valores)
```

«» R

  pégamelos aquí

y  te ajusto:

- si hay desalineación
-  cómo limpiar perfecto
- y  lo dejamos listo para graficar sin errores 

     ...

Ya hice esto: # Iniciamos sesión de Chrome y abrir visor
navegador <- ChromoteSession\$new() # Me permite
hacer la inspección de elementos
navegador\$view()

```
# Navegamos la URL dinámica
url_banrep <-
"https://suameca.banrep.gov.co/estadisticas-
economicas/informacionSerie/241/tasas_interes_indicador
_bancario_referencia_ibr"
```

```
navegador$page$navigate(url_banrep)
```

```
Sys.sleep(40)
```

```
# Extraemos el código fuente DOM actual y procesarlo
html_dinamico <- navegador$Runtime$evaluate(
  "document.documentElement.outerHTML"
)$result$value
```

```
html_doc <- read_html(html_dinamico)
```

Pero no sé qué más debo hacer para que funcione. No
está funcionando

Bien—ya hiciste la parte difícil (renderizar con `Chromote`). Si “no está funcionando”, casi
siempre es porque **el selector no coincide con cómo Oracle está pintando la grilla** o
porque **aún no cargó la data**.

Voy a darte un flujo de depuración que sí converge.

1) Verifica que la tabla exista en el DOM

Antes de extraer nada, confirma que **la grilla está presente**:

```
library(rvest)
```

«» R

- 0 → no cargó (espera/interacción)
- >0 → sí hay grilla (pasas al paso 2)

2) Evita IDs “largos” (son inestables)

En tu referencia usan cosas como:

```
«» R
```

Eso en IBR casi seguro cambia. Usa clases:

```
celdas <- html_doc %>%
  html_nodes(".oj-datagrid-cell span") %>%
  html_text(trim = TRUE)

filas <- html_doc %>%
  html_nodes(".oj-datagrid-row-header-cell") %>%
  html_text(trim = TRUE)
```

3) Diagnóstico obligatorio (esto te dice dónde falla)

```
length(celdas)
length(filas)
head(celdas, 20)
```

Interpreta:

- `length(celdas) = 0` → selector mal o no cargó
- `length(celdas)` pequeño (ej. 50) → solo cargó lo visible (scroll)
- valores mezclados (texto raro) → selector muy amplio

4) Si NO hay datos → fuerza render + scroll

Oracle carga por partes. Haz esto:

```
# scroll grande
navegador$Runtime$evaluate("
window.scrollTo(0, document.body.scrollHeight);
")

Sys.sleep(10)

# vuelve a capturar HTML
html_dinamico <- navegador$Runtime$evaluate(
  "document.documentElement.outerHTML"
)$result$value
```

```
html_doc <- read_html(html_dinamico)
```

Y repites extracción.

5) Si sigue en 0 → la tabla está dentro de iframe

Esto pasa mucho en SUAMECA.

Prueba:

```
iframes <- html_nodes(html_doc, "iframe")

<> R
```

Si ves algo como `/reporte-oac.html?... :`

👉 tienes que navegar **directamente a ese src**

```
navegador$Page$navigate("URL_DEL_IFRAME")

<> R
```

Y luego repites todo.

6) Alternativa más robusta (cuando Oracle molesta)

En vez de `.oj-datagrid-cell`, usa algo más general:

```
texto <- html_doc %>%
  html_nodes("span") %>%

<> R
```

Luego filtras con regex (tasas tipo `\\d+\\.\\d+`).

7) Señal de que ya lo lograste

Cuando esto funcione, verás:

```
head(celdas)
```

```
<> R
```



y

```
head(filas)
```

```
<> R
```



Lo que probablemente te está pasando

Por experiencia con esa página del [Banco de la República de Colombia](#):

👉 80% de probabilidad:

- estás leyendo el **contenedor vacío**
- la tabla real está en **iframe** o carga diferida



Para resolverlo rápido

Haz esto y pégamelo:

```
length(html_nodes(html_doc, ".oj-datagrid"))
```

```
<> R
```



Con eso te digo exactamente:

- si es iframe
- qué selector usar
- o cómo extraerlo directo sin ensayo-error

y lo dejamos funcionando en el siguiente mensaje 👍



si, fechas dice character (empty)

Perfecto, ese diagnóstico es clarísimo:



fechas = character(0) ⇒ **no estás leyendo la tabla real**

Eso pasa casi siempre en esa página del [Banco de la República de Colombia](#) porque:

👉 la tabla está dentro de un **iframe**

👉 tú estás parseando el HTML "externo" (vacío)



SOLUCIÓN (esto es lo que te falta)



1. Extraer el iframe

Corre esto:

```
iframes <- html_doc %>%  
  html_nodes("iframe") %>%  
  html_attr("src")
```

```
iframes
```

👉 Te va a devolver algo tipo:

```
<> R
```



2. Construir URL completa

```
iframe_url <- paste0("https://suameca.banrep.gov.co", iframes[1])
```

```
<> R
```



3. Navegar al iframe (CLAVE)

```
navegador$page$navigate(iframe_url)
```

```
</> R
```



4. Volver a extraer HTML (ahora sí el bueno)

```
html_dinamico <- navegador$Runtime$evaluate(  
  "document.documentElement.outerHTML"  
)$result$value
```

```
html_doc <- read_html(html_dinamico)
```



5. Ahora sí prueba los selectores

```
valores <- html_doc %>%  
  html_nodes(".oj-datagrid-cell span") %>%  
  html_text(trim = TRUE)  
  
fechas <- html_doc %>%  
  html_nodes(".oj-datagrid-row-header-cell") %>%  
  html_text(trim = TRUE)
```

6. Verifica

```
length(valores)  
length(fechas)  
head(valores)
```

```
</> R
```



 Aquí YA deberías ver datos reales.

Qué pasó (para que lo entiendas bien)

La página original:

```
informacionSerie/241
```



 solo carga un contenedor

El contenido real:

- está en /reporte-oac.html
- se renderiza dentro de iframe

Resumen claro

Tu error no es de código, es de arquitectura web:

Problema	Solución
HTML vacío	ir al iframe
fechas vacías	no estás en el DOM correcto
Oracle Analytics	siempre usa iframe

Siguiendo el paso

Cuando corras esto, pégame:

```
head(valores)
```

```
</> R
```



y te dejo:

- dataframe limpio
- conversión correcta
- listo para graficar

Sin más debugging 



intenté y no, no están en el nodo "span"


```

> celdas <- html_doc %>%
+   html_nodes("span") %>%
+   html_text(trim = TRUE)
> head(celdas, 50)
[1] ""
[2] ""
[3] ""
[4] "Leonardo Villar"
[5] "Bibiana Taboada"
[6] "Mauricio Villamizar"
[7] "Olga Lucía Acosta"
[8] "Laura Moisés"
[9] "César Giraldo"
[10] "Intervención del Gerente general del Banco de la
República en la Comisión Tercera del Senado"
[11] "El Banco de la República, presente en la Feria
Internacional del Libro de Bogotá 2026 (FILBo)"
[12] "¡Cuidado! Circula publicación falsa en redes sociales
relacionada con el Banco de la República"
[13] "Minutas de la reunión:"
[14] "Actualización trimestral del Reporte de Estabilidad
Financiera – abril de 2026"
[15] "Encuesta de percepción sobre riesgos del sistema
financiero - segundo semestre de 2025"
[16] "Reporte de la situación del crédito en Colombia -
diciembre de 2025"
[17] "Informe sobre la evolución reciente del
endeudamiento externo de los bancos colombianos -
diciembre de 2025"
[18] "Jueves, 29 de enero de 2026\nEl Banco de la
República informa cambios en la prestación de servicios de
Tesorería durante febrero de 2026"
[19] "Viernes, 23 de enero de 2026\nNuevas direcciones
para el cambio de efectivo en Ibagué, Montería, Riohacha,
Pasto y Quibdó"
[20] "Miércoles, 3 de diciembre de 2025\nLos
establecimientos de crédito deberán efectuar cambio y
provisión de billetes de baja denominación y monedas con
sus clientes"
[21] "Lunes, 1 de septiembre de 2025\nEl Banco de la
República lanza nuevo Sistema de agendamiento de citas
para los servicios de Tesorería"

```

[22] "Martes, 26 de agosto de 2025\nEn Quibdó, el Banco de la República informa sobre los puntos de cambio de efectivo, los elementos de seguridad de los billetes y todo lo que debe saber sobre la llegada de Bre-B"

[23] "Miércoles, 20 de agosto de 2025\nEste 10 de septiembre habrá modificación de horario en la ventanilla de Tesorería en Bogotá"

[24] "Viernes, 11 de julio de 2025\nEl Banco de la República presentó hoy la moneda conmemorativa del quinto centenario de fundación de la ciudad de Santa Marta"

[25] "Jueves, 3 de julio de 2025\nConozca los horarios de atención de este 23 de julio de 2025"

[26] "Lunes, 13 de abril de 2026\nTras seis meses de operación, Bre-B acumula más de 34 millones de usuarios registrados"

[27] "Sábado, 11 de abril de 2026\nValora Analitik: "Bre-B prepara su segunda fase: BanRep quiere incluir fiduciarias y comisionistas en el sistema de pagos inmediatos""

[28] "Martes, 7 de abril de 2026\nDespués del despegue, el gran reto de Bre-B es hacer que los salarios y las remesas también se vuelvan 'inmediatos'"

[29] "Lunes, 6 de abril de 2026\nkienyke: Bre-B ha movilizado más de \$100 billones en seis meses de operación"

[30] "Lunes, 6 de abril de 2026\n¡Envíe su información a tiempo! los titulares de cuentas de compensación podrán transmitir el Informe de Movimientos de las operaciones de marzo de 2026 hasta el jueves 30 de abril de 2026"

[31] "Lunes, 6 de abril de 2026\nPódcast BanRep | ¡Bre-B cumple seis meses de operación!"

[32] "Lunes, 6 de abril de 2026\nBre-B celebra sus primeros seis meses de operación plena, consolidándose como el motor de la modernización financiera en Colombia"

[33] "Viernes, 20 de marzo de 2026\nNuevas medidas regulatorias en Bre-B"

[34] "Pulso Económico Regional - I trimestre de 2026"

[35] "Intervención del Gerente general del Banco de la República en la Comisión Tercera del Senado - 22 de abril de 2026"

[36] "Respuesta a cuestionario Comisión Cuarta Constitucional Permanente de Cámara de Representantes"

[37] "Actualización trimestral del Reporte de Estabilidad Financiera – abril de 2026"

[38] "Jueves, 12 de febrero de 2026\nDivulgación de la encuesta de percepción sobre riesgos del sistema financiero - II semestre de 2025"

[39] "Miércoles, 11 de febrero de 2026\nDivulgación de la encuesta trimestral sobre la situación del crédito en Colombia - IV trimestre de 2025"

[40] "Martes, 10 de febrero de 2026\nDivulgación de la encuesta de la evolución reciente del endeudamiento externo de los bancos colombianos - IV trimestre de 2025"

[41] "Retornar a"

[42] "Estadísticas económicas"

[43] "Tasas de interés"

[44] "Participante"

[45] "Tasa nominal"

[46] "Participante"

[47] "Tasa nominal"

[48] "Participante"

[49] "Tasa nominal"

[50] "Participante"

Perfecto—esto ya nos dice exactamente qué está pasando:

- Estás leyendo el **HTML equivocado (landing + headers)**
- Aún **NO** estás dentro del visor de Oracle (la tabla real)

Lo que ves (nombres, noticias, "Participante") confirma que estás en la capa superficial del sitio del Banco de la República de Colombia, no en la grilla.

🧠 Problema real (sin rodeos)

Esa URL:

informacionSerie/241



👉 no contiene los datos

👉 solo incrusta un visor dinámico (Oracle Analytics)

Por eso:

- `span` no funciona ❌

- `.oj-datagrid` no aparece ❌

✅ SOLUCIÓN CORRECTA (la que sí funciona)

👉 Tienes que ir **directamente** al visor Oracle, igual que en el ejemplo de clase.

🔗 1. Usa la URL "back" (esto es clave)

Prueba esta estructura (adaptada a IBR):

```
url_hannon <- "https://suamara.hannon.gov.co/estadisticas-economicas-back/reporte-oac.html?n="
</> R
```



👉 Nota:

- Esto es lo mismo que usaron en IED
- pero apuntando al módulo de IBR

⚠️ Si esa URL no carga (puede pasar)

Entonces haz esto (método seguro):

🔍 Encuentra la URL real del visor

1. Abre la página en navegador
2. Presiona F12
3. Ve a **Network**
4. Filtra por:
 - reporte-oac
5. Recarga la página
6. Copia la URL que aparece

👉 Esa ES la que debes usar en Chrome

🚀 2. Navegar (reemplaza tu URL actual)

</> R



3. Verificación clave

Ahora esto Sí debería funcionar:

```
html_dinamico <- navegador$Runtime$evaluate(  
  "document.documentElement.outerHTML"  
)$result$value  
  
html_doc <- read_html(html_dinamico)  
  
html_nodes(html_doc, ".oj-datagrid") |> length()
```

👉 Si esto > 0 → ya estás en la tabla correcta ✅

4. Extraer datos (ahora sí)

```
valores <- html_doc %>%  
  html_nodes(".oj-datagrid-cell span") %>%  
  html_text(trim = TRUE)  
  
fechas <- html_doc %>%  
  html_nodes(".oj-datagrid-row-header-cell") %>%  
  html_text(trim = TRUE)
```

Conclusión (directa)

- ❌ Tu código está bien
- ❌ Tu problema NO es R
- 🔴 Tu problema es la URL

👉 Estás scrapeando la página equivocada

🚀 Si quieres resolverlo en 5 min

👉 Haz esto y mándamelo:

</> R



o screenshot del **Network**

👉 y 👉 te doy la URL exacta correcta del visor

👉 + 👉 selectores listos

Sin más vueltas 👍



me di cuenta que lo que debo pulsar para scrapear es el botón de "vista de tabla". Porque ahí si aparece. Entonces debería usar un código como esto?:# 3.3 Inyectar JavaScript para interactuar con la interfaz (Clic automáticos en la pestaña 'Anual')

codigo_js <- "

```
{  
  let elementos =  
    Array.from(document.querySelectorAll('*'));  
  let pestana_anual = elementos.find(el =>  
    el.textContent.trim() === 'Anual');
```

```
  if(pestana_anual) {  
    pestana_anual.click();  
    'Clic enviado con éxito';  
  } else {  
    'No se encontró la pestaña Anual';  
  }  
}
```

pero para el botón de tabla?

https://suameca.banrep.gov.co/estadisticas-economicas/informacionSerie/241/tasas_interes_indicador_bancario_referencia_ibr

Ahora sí diste en el punto clave 🍌

👉 Exactamente eso es lo que te faltaba: activar la *vista de tabla*.

🧠 Qué está pasando

En esa página del **Banco de la República de Colombia**:

- Por defecto → **vista gráfica (Highcharts)**
- Los datos están en:
 - `<table class="highcharts-data-table">` ✓
- PERO esa tabla:
 - ✗ no existe hasta que haces clic en “Vista de tabla”

✅ Sí: necesitas hacer clic con JavaScript

Tu lógica es correcta. Solo cambia el target:

🔧 Código adaptado (botón “Vista de tabla”)

```
codigo_js <- "  
{  
  let elementos = Array.from(document.querySelectorAll('button, a, div'));  
  let boton_tabla = elementos.find(el =>  
    el.textContent && el.textContent.trim().includes('Vista de tabla')  
  );  
  
  if(boton_tabla) {  
    boton_tabla.click();  
    'Clic en Vista de tabla enviado';  
  } else {  
    'No se encontró el botón Vista de tabla';  
  }  
}
```

```
}  
}  
"
```

```
navegador$Runtime$evaluate(codigo_js)
```

```
</> R
```



🔍 Luego extraes HTML otra vez

```
html_dinamico <- navegador$Runtime$evaluate(  
  "document.documentElement.innerHTML"  
)$result$value  
  
html_doc <- read_html(html_dinamico)
```



✅ Ahora sí cambia TODO: scraping mucho más fácil

Ya NO uses `.oj-datagrid`

👉 ahora tienes una tabla HTML normal

✂ Extraer tabla directamente

```
tabla <- html_doc %>%  
  html_node(".highcharts-data-table") %>%
```

```
</> R
```



🔧 Verifica

```
</> R
```



👉 Deberías ver algo como:

Periodo	IBR
2020-01-02	4.25
...	...

 Limpieza

```
library(dplyr)

df_ibr <- tabla %>%
  rename(
    fecha = 1,
    tasa = 2
  ) %>%
  mutate(
    fecha = as.Date(fecha),
    tasa = as.numeric(tasa)
  )
```

 Si no funciona el click

A veces el texto no coincide exacto. Alternativa más robusta:

```
codigo_js <- "
document.querySelectorAll('button').forEach(btn => {
  if(btn.innerText.includes('tabla')) {
    btn.click();
  }
});
"
```

 Conclusión (importante)

Acabas de pasar de:

- ✗ scraping complicado (Oracle grid)
- a:

- ✓ scraping trivial (HTML table)

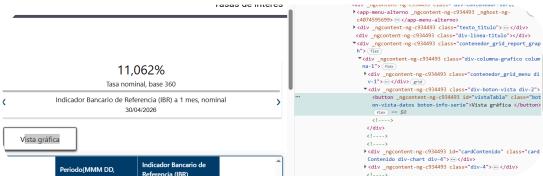
👉 Esta es la mejor estrategia posible para este caso

 Siguiente paso

Si quieres, te ayudo a:

- ↳ filtrar exactamente desde 2020
- ↳ unir con SOFR
- y ↳ hacer el gráfico perfecto (con todos los requisitos del profe)

Solo dime 👍

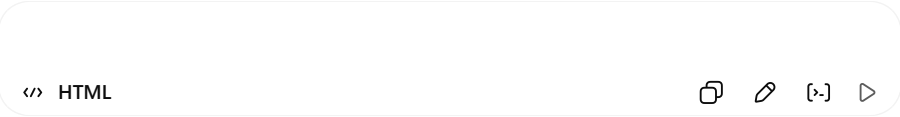


no funcionó. pero esto es lo que da cuando inspecciono el botón de Vista tabla:

Perfecto—este screenshot ya revela **exactamente el problema** y también la solución limpia.

 Qué estás viendo realmente

Ese botón:



👉 OJO:

- El id es vistaTabla
- Pero el texto dice “Vista gráfica”

Eso significa:

- El botón es un *toggle* (interruptor)

- Si estás en gráfica → botón dice "Vista tabla"
- Si estás en tabla → botón dice "Vista gráfica"

👉 Es decir: **no puedes confiar en el texto** ❌

👉 Debes usar el ID ✅

✅ SOLUCIÓN CORRECTA (robusta)

Haz clic directamente por ID:

```
codigo_js <- "  
document.getElementById('vistaTabla').click();
```

</> R



```
navegador$Runtime$evaluate(codigo_js)
```

</> R



🔍 Verificación clave (no te la saltes)

Después del click:

```
html_dinamico <- navegador$Runtime$evaluate(  
  "document.documentElement.outerHTML"  
)$result$value
```

```
html_doc <- read_html(html_dinamico)
```

```
html_nodes(html_doc, "table") |> length()
```

👉 Si > 0 → ya tienes tabla ✅

🔗 Extraer tabla (ahora sí sencillo)

Por lo que se ve en tu inspector:

</> HTML



Entonces:

```
tabla <- html_doc %>%  
  html_node(".highcharts-data-table") %>%
```

</> R



🔧 Diagnóstico si aún falla

Caso 1: sigue sin aparecer tabla

👉 El click no se ejecutó bien

Solución más agresiva:

```
codigo_js <- "  
let btn = document.getElementById('vistaTabla');  
if(btn){  
  btn.click();  
  btn.click(); // doble click por seguridad  
}  
"
```

Caso 2: el botón aún no existe al ejecutar

👉 Espera a que cargue completamente:

</> R



o mejor:

```
navegador$Runtime$evaluate("  
  setTimeout(() => {  
    document.getElementById('vistaTabla').click();  
  }, 3000);
```

</> R



🎯 Conclusión (esto es lo importante)

- ❌ Tu intento falló porque buscabas por texto
- ✅ La página usa **toggle dinámico**
- ✅ La forma correcta es **usar el ID** (`vistaTabla`)

🚀 Siguiendo paso

Cuando hagas:

</> R



👉 pégamelo aquí

y te dejo:

- limpieza perfecta
- filtrado desde 2020
- y el gráfico final listo para entregar

Ya estás a nada de terminar 🍷



```
sale esto: > codigo_js <- "  
+ document.getElementById('vistaTabla').click();  
+ "  
> navegador$Runtime$evaluate(codigo_js)  
$result  
$result$type  
[1] "undefined"
```

```
> html_dinamico <- navegador$Runtime$evaluate(  
+ "document.documentElement.innerHTML"  
+ )$result$value
```

```
>  
> html_doc <- read_html(html_dinamico)  
> html_nodes(html_doc, "table")  
{xml_nodeset (7)}  
[1] <table class="table" style="width:100%;" border="1"  
cellpadding="1" cellspacing="1">\n<thead> <tr> <th  
colspan="3"> ...  
[2] <table tabindex="-1" summary="Table representation  
of chart." id="highcharts-data-table-0">\n<caption  
class="high ...  
[3] <table _ngcontent-ng-c934493=""> <tbody  
_ngcontent-ng-c934493="">\n<tr _ngcontent-ng-  
c934493="" class="tr-columna- ...  
[4] <table _ngcontent-ng-c934493=""> <tbody  
_ngcontent-ng-c934493="">\n<tr _ngcontent-ng-  
c934493="" class="tr-columna- ...  
[5] <table _ngcontent-ng-c934493=""> <tbody  
_ngcontent-ng-c934493="">\n<tr _ngcontent-ng-  
c934493="" class="tr-columna- ...  
[6] <table _ngcontent-ng-c934493=""> <tbody  
_ngcontent-ng-c934493="">\n<tr _ngcontent-ng-  
c934493="" class="tr-columna- ...  
[7] <table _ngcontent-ng-c934493=""> <tbody  
_ngcontent-ng-c934493="">\n<tr _ngcontent-ng-  
c934493="" class="tr-columna- ...  
>
```

Perfecto—esto ya está **prácticamente resuelto**.

👉 El click **sí funcionó**, aunque `evaluate` devuelva `"undefined"` (eso es normal en JS).

La evidencia clave es esta:

</> R



👉 🎯 Esa es TU tabla correcta (la que genera Highcharts).

✅ Qué debes hacer ahora

🔥 1. Extraer ESA tabla (no todas)

```
tabla <- html_doc %>%
  html_node("#highcharts-data-table-0") %>%
```

```
</> R
```



2. Verifica

```
</> R
```



Deberías ver algo como:

Periodo	IBR
2009-05-18	5.892
...	...

Importante (esto explica por qué antes no funcionaba)

- Hay 7 tablas en el DOM
- Solo 1 es la buena
- Las demás son:
 - layout ❌
 - menús ❌
 - componentes Angular ❌

Por eso `html_nodes("table")` te confundía

3. Limpieza final

```
library(dplyr)
library(stringr)

df_ibr <- tabla %>%
  rename(
    fecha = 1,
```

```
tasa = 2
) %>%
mutate(
  fecha = as.Date(fecha),
  tasa = str_replace_all(tasa, ",", "."),
  tasa = as.numeric(tasa)
)
```

4. Filtrar lo que te pide el ejercicio

```
df_ibr <- df_ibr %>%
  filter(fecha >= as.Date("2020-01-01"),
```

```
</> R
```



Nota importante (esto puede aparecer)

Si `tabla` sale vacía o rara:

es porque el click tardó en renderizar

Solución:

```
</> R
```



antes de extraer el HTML.

Conclusión

Ya lograste lo difícil:

- ✓ Navegación dinámica
- ✓ Activar vista correcta
- ✓ Identificar tabla correcta

Lo que te faltaba era solo:

```
usar #highcharts-data-table-0
```


 Siguiente paso

Si quieres, ahora te ayudo a:

- ↳ unir con SOFR
- ↳ hacer el gráfico comparativo
- y ↳ escribir el análisis (literal B)

Solo dime 👍



```
> tabla_ibr <- html_doc %>%
+   html_node("#highcharts-data-table-0") %>%
+   html_table(fill = TRUE)
> head(tabla_ibr)
# A tibble: 6 × 6
  Periodo(MMM DD, AAAA) Indicador Bancario de R...1
Indicador Bancario d...2 Indicador Bancario d...3 Indicador
Bancario d...4
  <chr>                <dbl>                <dbl>
<dbl>                <dbl>
1 2008/01/02          8.95                NA
NA                    NA
2 2008/01/03          8.97                NA
NA                    NA
3 2008/01/04          8.95                NA
NA                    NA
4 2008/01/08          8.96                NA
NA                    NA
5 2008/01/09          8.97                NA
NA                    NA
6 2008/01/10          8.98                NA
NA                    NA
#  abbreviated names: 1 Indicador Bancario de
Referencia (IBR) overnight, nominal,
# 2 Indicador Bancario de Referencia (IBR) a 1 mes,
nominal,
# 3 Indicador Bancario de Referencia (IBR) a 3 meses,
nominal,
# 4 Indicador Bancario de Referencia (IBR) a 6 meses,
```

```
nominal
#  1 more variable: Indicador Bancario de Referencia
(IBR) a 12 meses, nominal <dbl>

Salió esto! Funcionó!!
```

Perfecto—esto ya está **correcto y completo a nivel de extracción**. Ahora toca dejarlo **usable para tu ejercicio** (IBR ON diaria desde 2020).

 Qué tienes ahora

Tu tabla incluye varias series:

- IBR **overnight (ON)**  ← la que necesitas
- IBR 1 mes
- IBR 3 meses
- IBR 6 meses
- IBR 12 meses

 Debes quedarte solo con overnight

 1. Seleccionar la serie correcta

Esa columna es:

Indicador Bancario de Referencia (IBR) overnight, nominal

```
library(dplyr)

df_ibr <- tabla_ibr %>%
  select(
    fecha = `Periodo(MMM DD, AAAA)` ,
    tasa = `Indicador Bancario de Referencia (IBR) overnight, nominal`
  )
```

 2. Limpiar datos

```
library(stringr)

df_ibr <- df_ibr %>%
  mutate(
    fecha = as.Date(fecha),
    tasa = as.numeric(tasa)
  )
```

👉 Aquí no necesitas reemplazar comas porque ya vino como db1 ✔

🎯 3. Filtrar periodo requerido

```
df_ibr <- df_ibr %>%
  filter(
    fecha >= as.Date("2020-01-01"),
    fecha <= as.Date("2025-10-31")
  )
```

🔧 4. Verificación final

```
summary(df_ibr)
```

<> R

👉 Debes ver:

- fechas desde 2020 ✔
- valores sin NA (o muy pocos) ✔

⚠ Nota técnica importante

Veo esto:

```
<> R
```

👉 Eso es normal:

- otras maturities no tienen datos en esas fechas

- pero overnight sí está completo

🎯 Resultado final esperado

Tu dataframe debe quedar así:

fecha	tasa
2020-01-02	4.25
...	...

🚀 Ya estás en la última etapa

Tienes:

- ✔ SOFR (API)
- ✔ IBR (scraping dinámico)

👉 Solo falta:

1. unir datasets
2. graficar
3. hacer análisis comparativo

👉 Si quieres cerrar el ejercicio

En el siguiente paso te doy:

- ↳ código de **ggplot** listo (con todos los requisitos del profe)
- y ↳ el **análisis escrito (literal B)** listo para entregar

Solo dime 👍

📄 👍 🗨 ⬆ ↺ ...

eso era todo. Gracias!

Perfecto, lo sacaste bien—esa era la parte realmente compleja del ejercicio.

Ya tienes:

- API funcionando (SOFR)
- scraping dinámico resuelto (IBR)
- datos en formato usable

Con eso, el resto (gráfica y análisis) es mecánico.

Si en algún punto te falla algo más adelante—por ejemplo:

- alinear fechas entre IBR y SOFR
- manejar días sin datos
- o detalles del gráfico que pide el profe

vuelve y lo vemos sin problema.

